

CSP: script-src unsafe-inline (Systemic)

Assessment Identifier: **WS-DYN-02** | Assessment Type: **Dynamic** | Date: **September 08, 2026**

Severity: MEDIUM	CVSS: 6.1	CWE: CWE-693
OWASP: A05:2021 - Security Misconfiguration	WASC ID: 15	Confidence: High
Target / Component: Documentation Route	Affected URL: https://www.worldmonitor.app/docs/documentation	Parameter: content-security-policy

1. Description

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. The application's Content Security Policy uses 'unsafe-inline' within the script-src directive, allowing inline scripts to execute. This significantly weakens the CSP protection mechanism against Cross-Site Scripting (XSS) attacks.

2. Steps to Reproduce

1. Send an HTTP GET request to https://www.worldmonitor.app/docs/documentation.
2. Inspect the HTTP response headers returned by the server.
3. Locate the content-security-policy header.
4. Verify that the header allows 'unsafe-inline' execution.

3. Controlled Proof of Concept

HTTP Header Evidence:


```

content-security-policy: worker-src * blob: data: 'unsafe-eval' 'unsafe-inline'; object-src data: ; base-uri 'self'; upgrade-insecure-requests; frame-ancestors 'self' https://dashboard.mintlify.com https://app.mintlify.com; form-action 'self' https://codesandbox.io;

```

Note: Because 'unsafe-inline' is present, any injected inline <script> tag or inline event handler (e.g., on load=, onerror=) will execute in the victim's browser if an XSS injection point exists.

4. Business & Security Impact

While a weak CSP is not an exploit on its own, specifying 'unsafe-inline' strips away the primary defense-in-depth layer designed to block cross-site scripting (XSS). If a parameter on the application becomes vulnerable to XSS, attackers can execute arbitrary JavaScript in the user's browser, leading to session hijacking, sensitive data theft, or unauthorized actions on behalf of authenticated users.

5. Remediation Recommendations

- Primary Fix:** Remove 'unsafe-inline' from the script-src and worker-src directives and adopt cryptographically secure nonces or hashes.
1. Remove 'unsafe-inline' from the script-src and worker-src directives.
 2. Refactor inline scripts into external .js files served from trusted origins.

3. If inline scripts are strictly required, implement cryptographically secure nonces (e.g., script-src 'nonce-rAnd0m123') or hashes (e.g., script-src 'sha256-...') for specific allowed scripts. Ensure configured servers and proxies strictly enforce valid CSP headers across all documentation routes.

6. Evidence & Validation

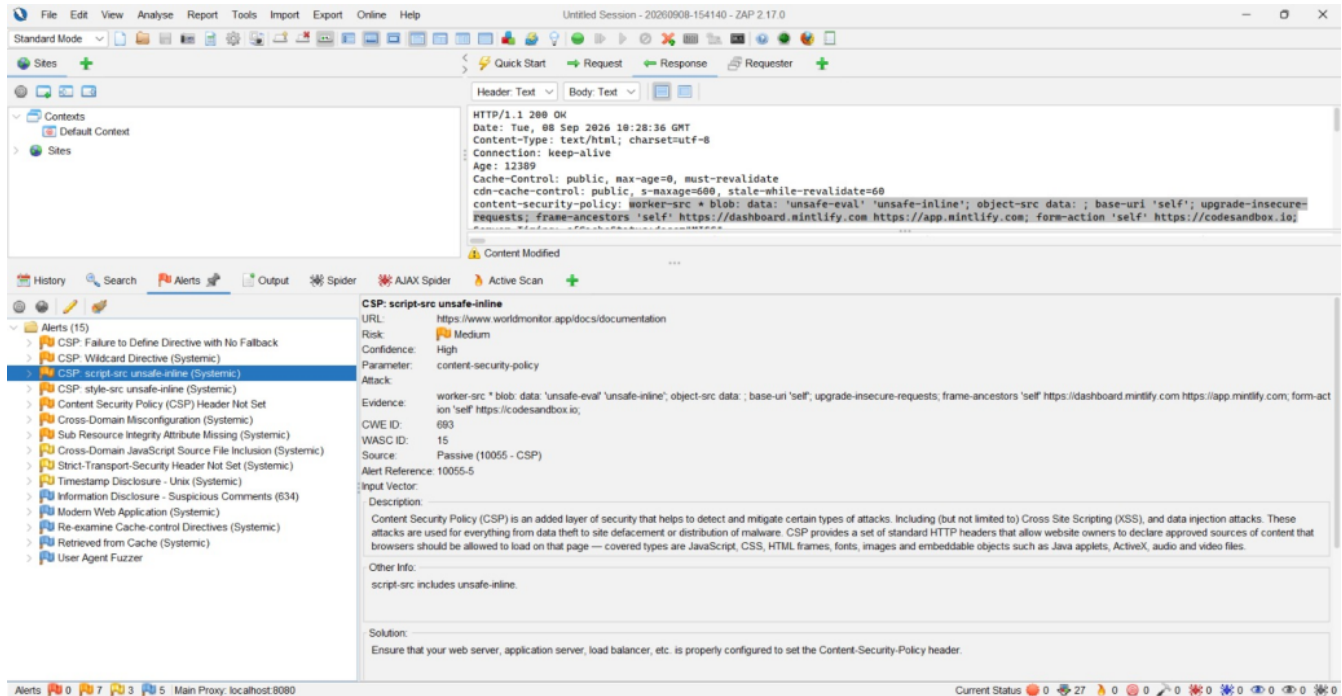


Figure 1: Matched Proof-of-Concept Screenshot for Vulnerability #2 (2/poc-1.png)

Authorization & Safety Compliance: This security assessment was conducted strictly under authorized parameters in accordance with ethical hacking standards. All validations and safe proof-of-concept tests were non-destructive. No persistent modifications or sensitive data exfiltration occurred.